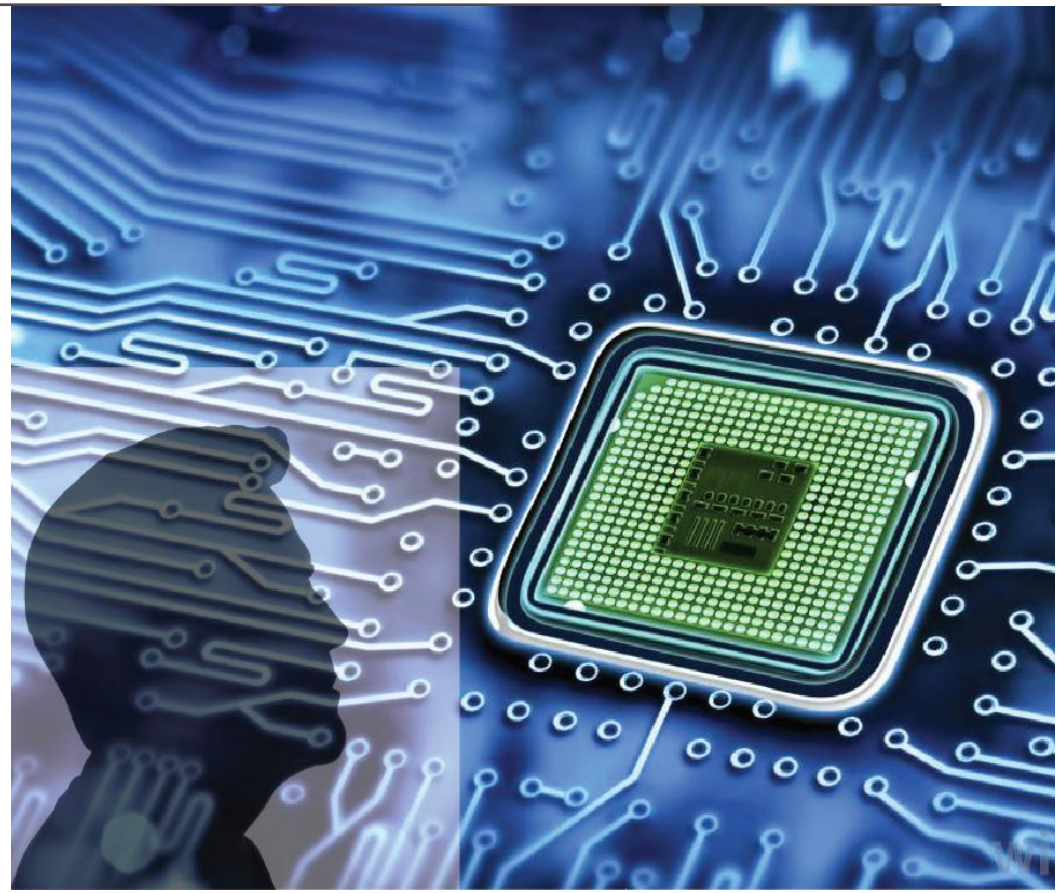
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-6 Lecture-2
High Level Synthesis



Sneh Saurabh
18th September, 2025

CDFG: Optimizations

- Traditional compiler optimizations
- Optimizations specific to hardware

Range Minimizations (Bit trimming)

- Based on ranges of primary inputs and primary outputs
- Based on operations with constants

a = b+c;

Assume

- Range of *b* is [0,4] bits
- Range of *c* is [0,3] bits
- *a* was declared as int

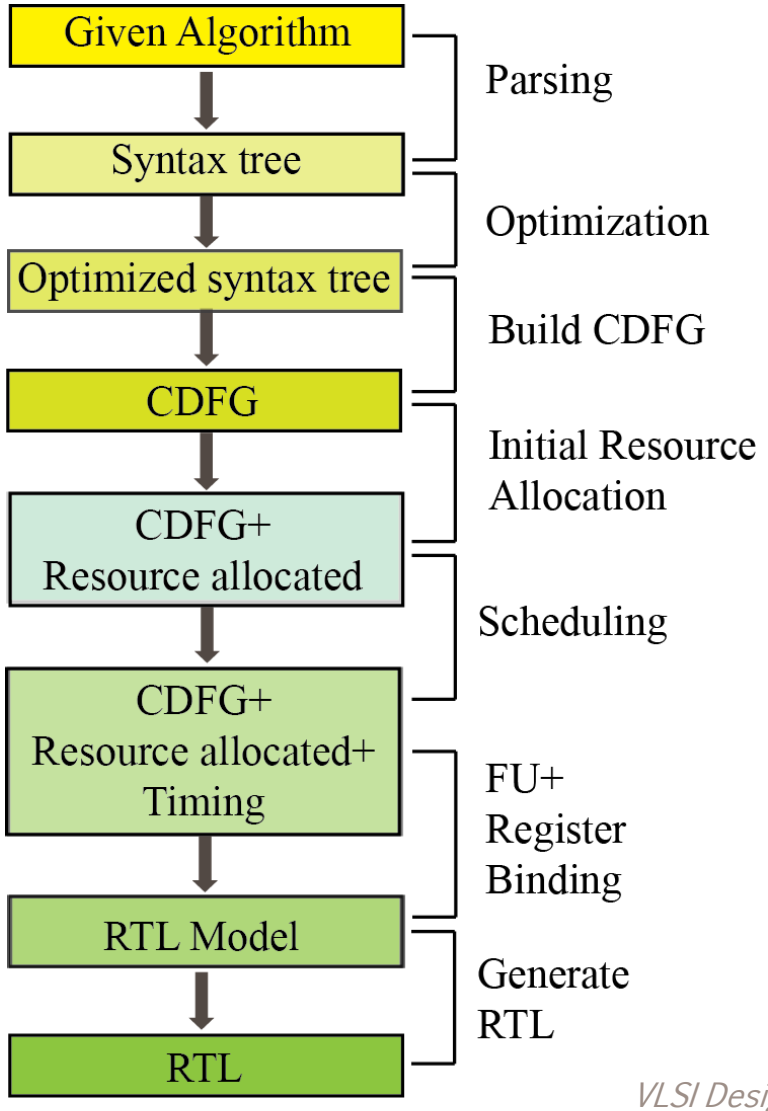
- Range of *a* can be reduced to [0,5] bits

Assume

- *b* can take a value between 1 to 10
- *c* can take a value between 1 to 5

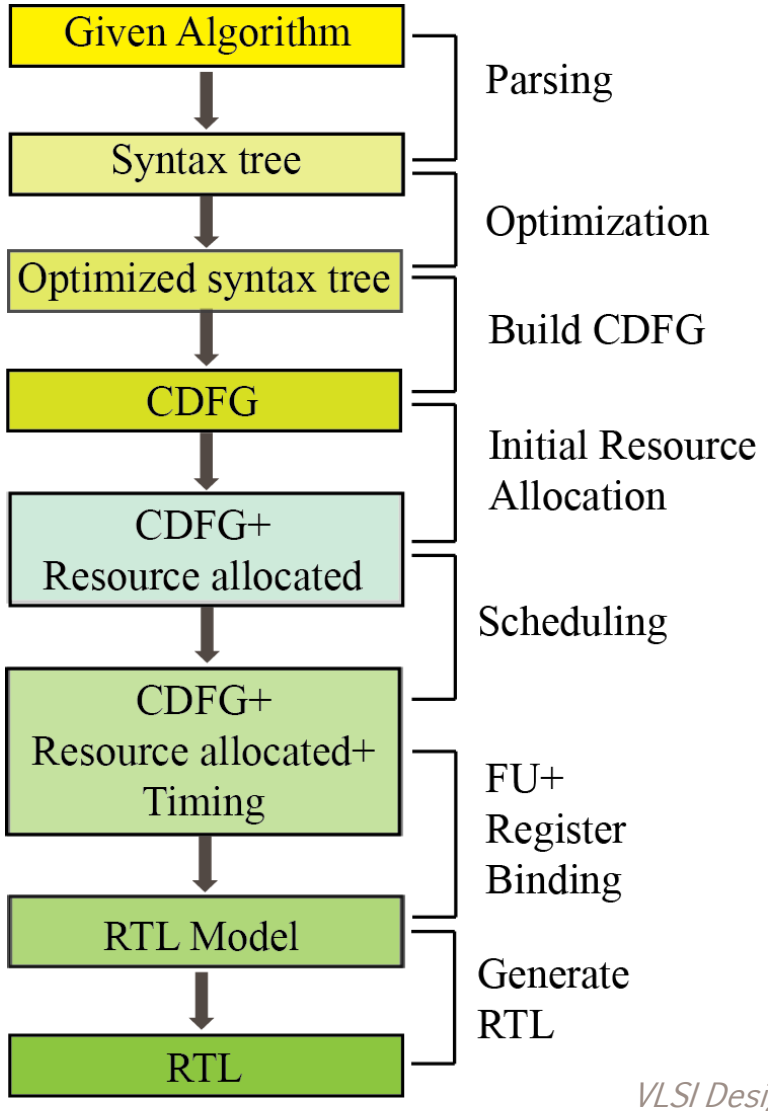
- *c* can take a value between 2 to 15
- Range of *a* can be reduced to [0,3] bits

HLS: Initial Resource Allocation



- Function Unit (FU) mapped to CDFG and subsequently modified
- For example: adder, multiplier etc. allocated

HLS: Scheduling



- Scheduling establishes "timing" to the given "algorithm"

- Resources needed in different cycles can be shared to improve area
- Parallelism can be exploited to improve timing
- QoR of HLS largely depends on scheduling algorithm

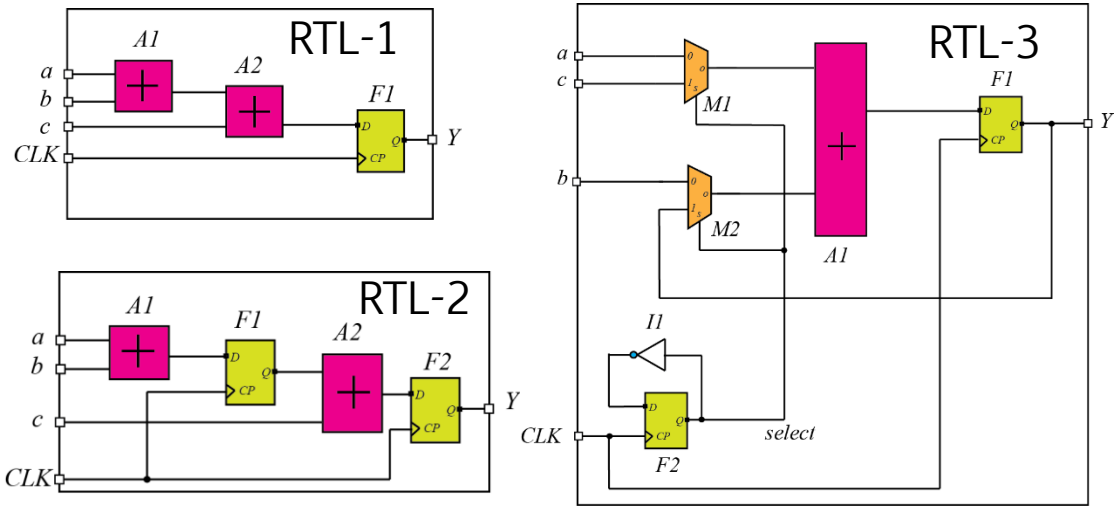
- Two main tasks:
  - Mapping operations to clock cycles
  - Mapping hardware instances to operations

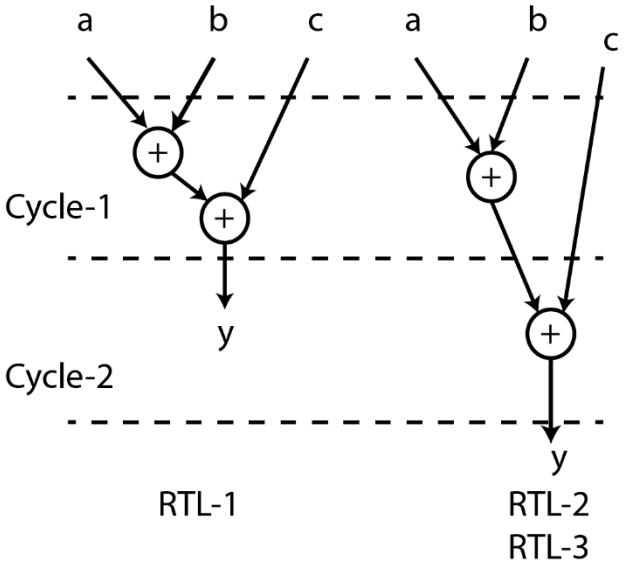
Flexibility

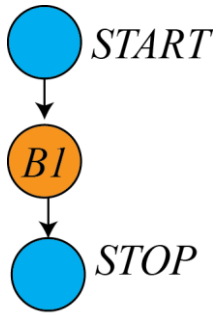
- FSM extracted from CFG can be modified (by adding new states)
- Available resources can be increased

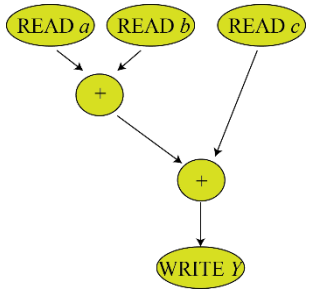
Scheduling: Mapping operations to Clock Cycles

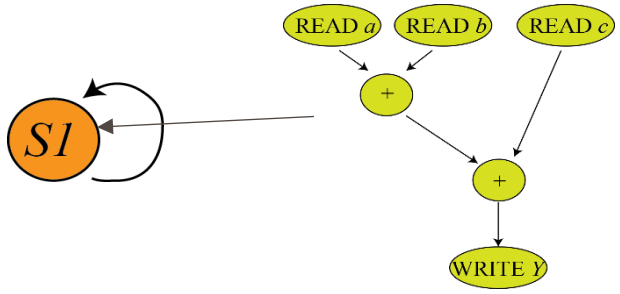
- Algorithmic behavior: $Y = a + b + c$

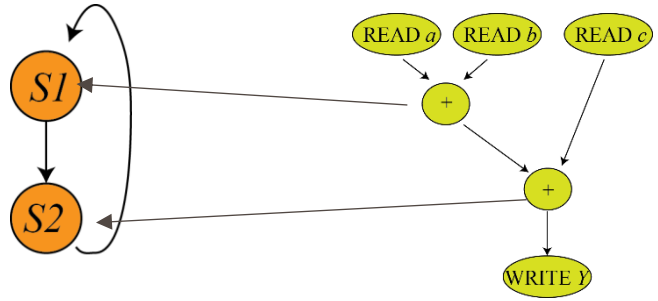




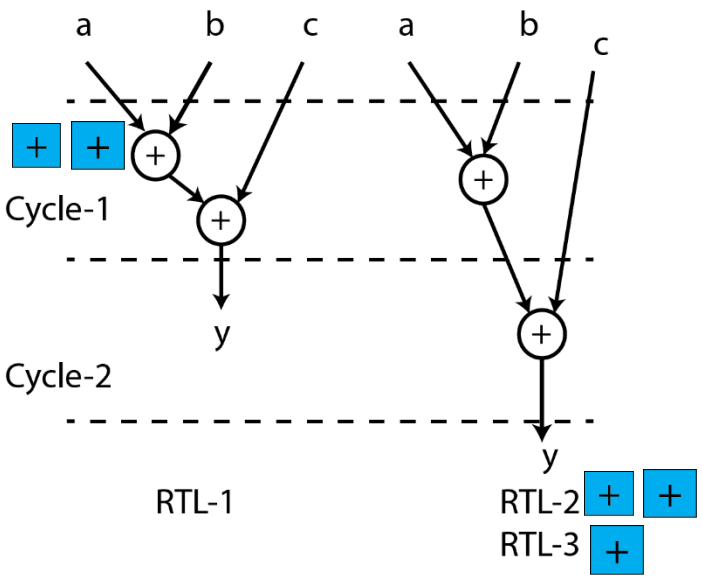


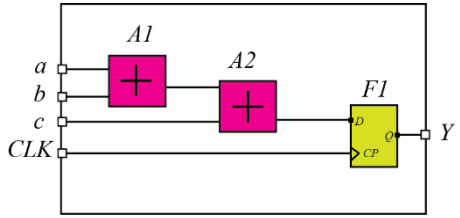






Scheduling: Mapping Hardware Instances to Operations



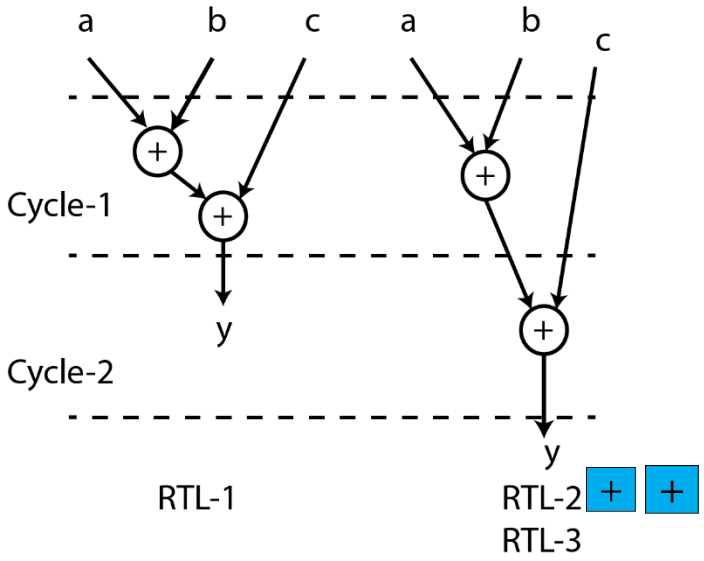


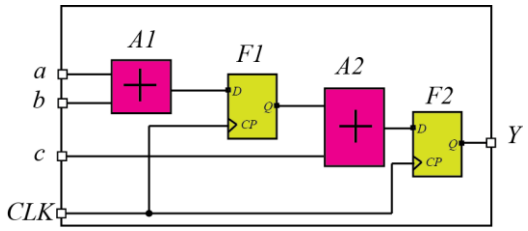
- Resources used in the same cycle cannot be shared

- Resources used in different cycles can be shared

```
module compute_parallel (
    input wire clk,             // Clock input
    input wire reset,           // Reset signal to re-initialize the result
    input wire [31:0] a, b, c, // Input operands
    output reg [31:0] Y         // Output result
);

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            Y <= 32'd0;             // Reset the output
        end else begin
            Y <= (a + b) + c;       // Perform both additions in parallel
        end
    end

endmodule
```

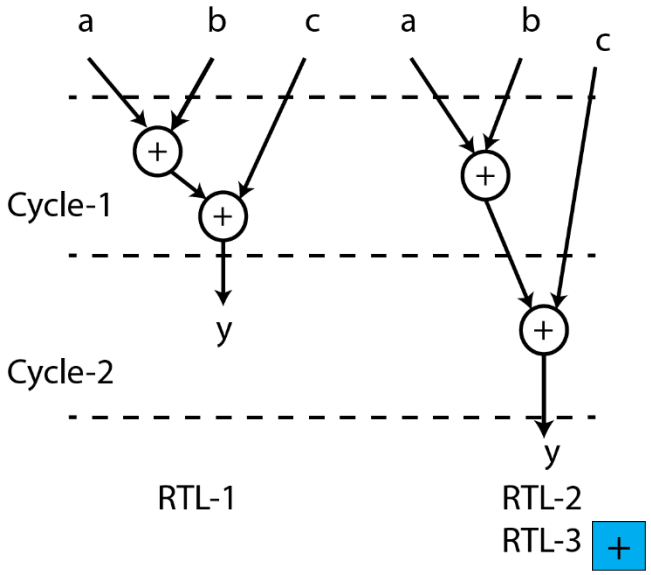
Scheduling: Mapping Hardware Instances to Operations

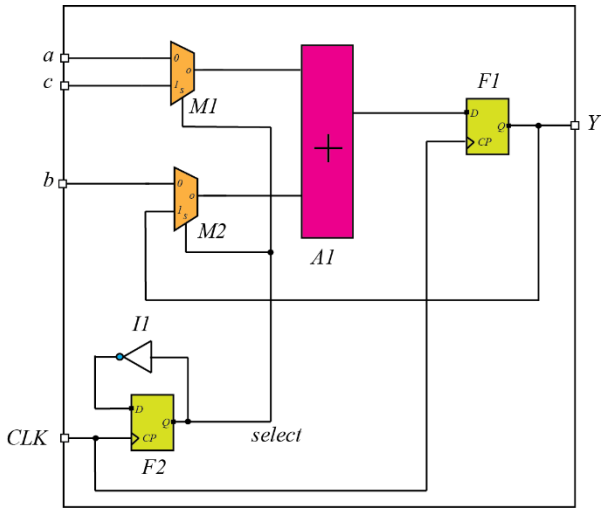





```verilog
module compute_pipeline (
    input wire clk,             // Clock input
    input wire reset,           // Reset signal to re-initialize the pipeline
    input wire [31:0] a, b, c, // Input operands
    output reg [31:0] Y         // Output result
);

    reg [31:0] stage1_result;  // Intermediate register for the result of a + b

    // Stage 1: Perform the first addition (a + b)
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            stage1_result <= 32'd0;    // Reset intermediate result
        end else begin
            stage1_result <= a + b;    // Perform first addition
        end
    end

    // Stage 2: Perform the second addition (stage1_result + c)
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            Y <= 32'd0;                // Reset final output
        end else begin
            Y <= stage1_result + c;    // Perform second addition
        end
    end

endmodule
```

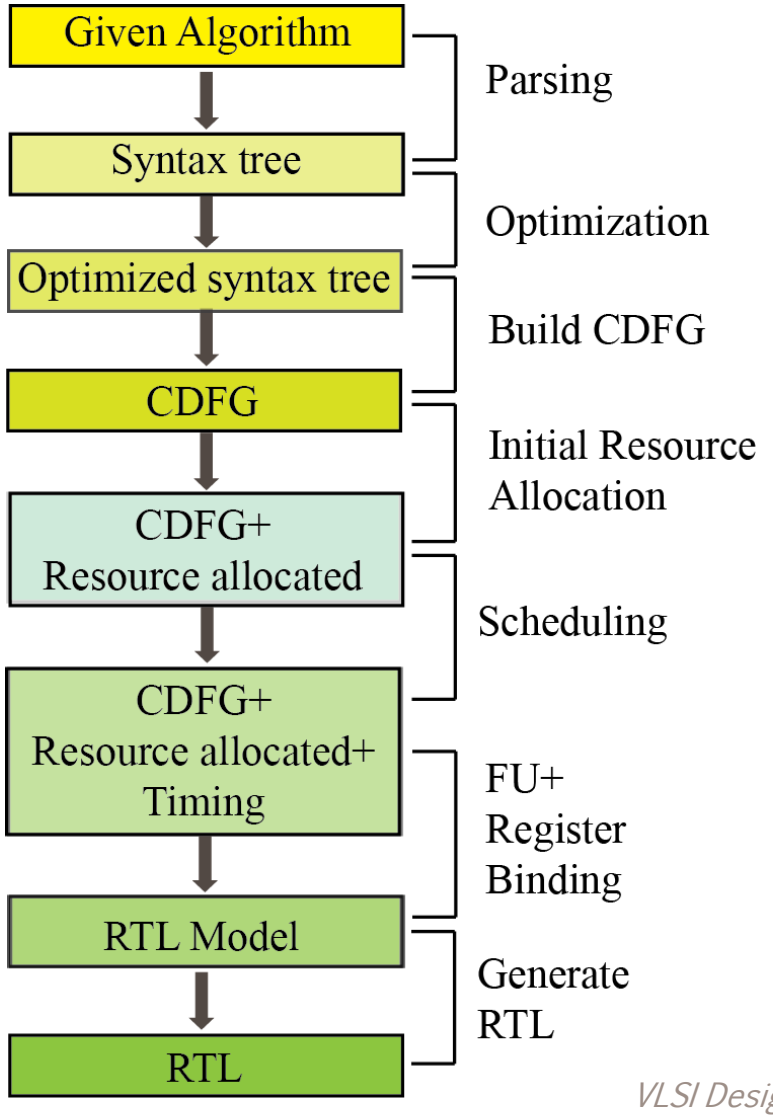
Scheduling: Mapping Hardware Instances to Operations



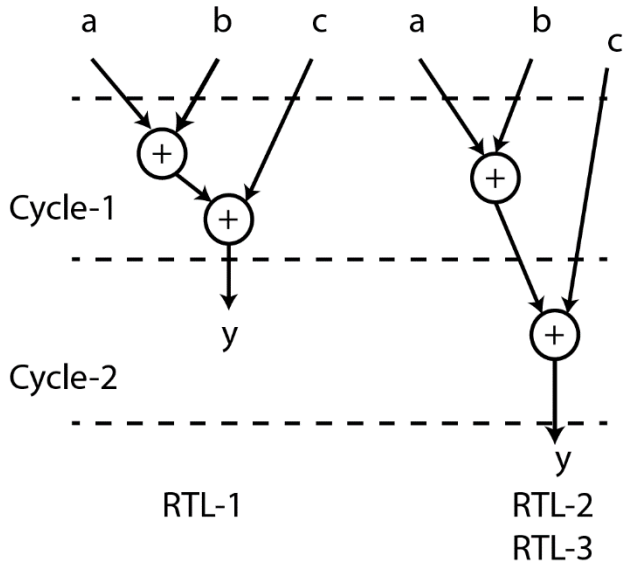



```
module compute (
    input wire clk,            // Clock input
    input wire reset,          // Reset signal to re-initialize the state machine
    input wire [31:0] a, b, c, // Input operands
    output reg [31:0] Y        // Output result
);

    reg [1:0] state;           // State register

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= 2'd0;     // Reset to initial state
        end else begin
            case (state)
                2'd0: begin
                    Y <= a + b;         // Perform first addition, store in Y
                    state <= 2'd1;      // Transition to next state
                end

                2'd1: begin
                    Y <= Y + c;         // Perform second addition with Y
                    state <= 2'd0;      // Return to idle state
                end
            endcase
        end
    end

endmodule
```

HLS: Function Unit and Register Binding

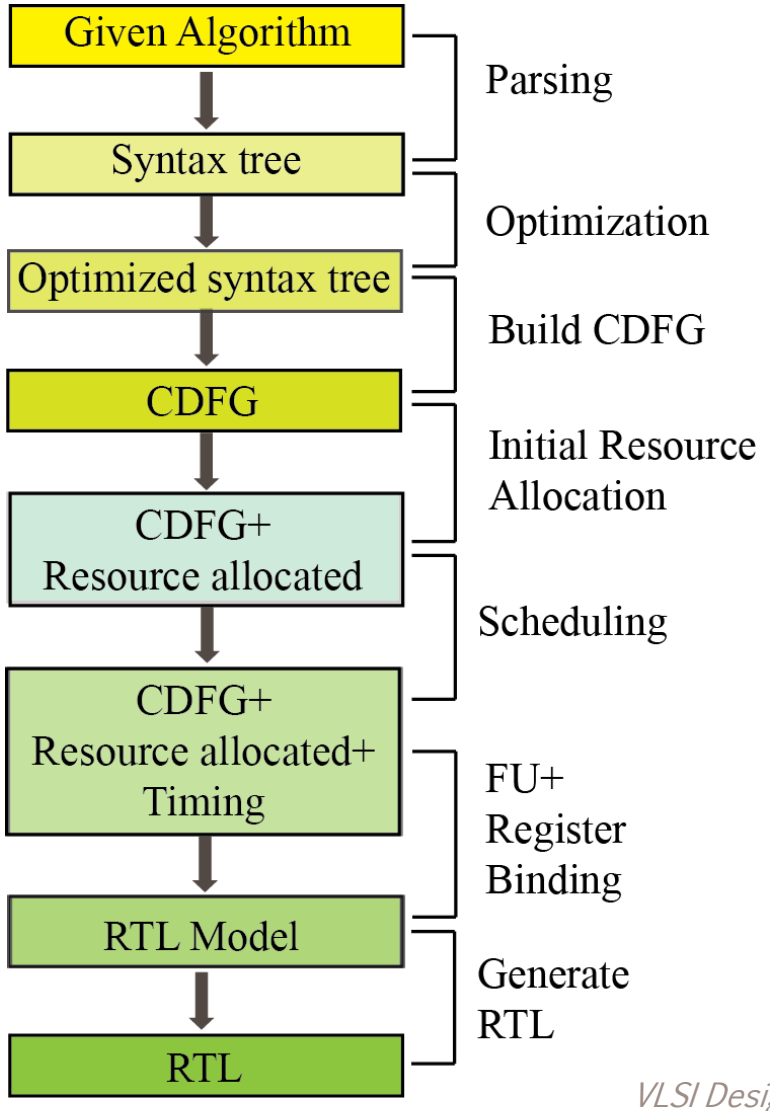


- Based on scheduling and computation required, appropriate FU from the library are assigned
  - Different FU binding might give different wiring/multiplexing circuitry
  - Optimization may be performed to improve timing, resource utilization, or other metrics



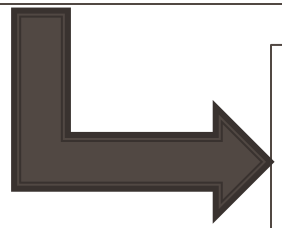
- Registers are assigned where the result of computation performed in one cycle is used in another cycle
- Sometimes sharing is possible if lifespan intervals do not overlap

HLS: Generate RTL



- Control Path is represented as Finite State Machine (FSM)
- Data Path and Control Path are converted to suitable RTL constructs and written in Verilog/VHDL
- RTL can be represented in several ways to allow further optimization down the flow (RTL to GDS flow)

High-level Synthesis



Optimizations

Handling Loops

- **Loop Unrolling:** can expose other opportunities of optimization

```
for (int i=0; i<N; i++) {
  out[i] = p[i]*s[i] + p[i+2]*s[i+2];
}
```

- Number of memory reads = 4N=16
- Number of multiplication =2N=8

```
out[0] = p[0]*s[0] + p[2]*s[2];
out[1] = p[1]*s[1] + p[3]*s[3];
out[2] = p[2]*s[2] + p[4]*s[4];
out[3] = p[3]*s[3] + p[5]*s[5];
```

Number of memory reads = 2(N+2)=12
Number of multiplication =N+2=6

- **Loop fusion:** combines two adjacent loops that iterate same number of times into a single loop (when there are no dependencies between them)

```
for (int i=0; i<N; i++) {
  sum[i] = A[i]+B[i];
}

for (int i=0; i<N; i++) {
  diff[i] = A[i]-B[i];
}
```

```
for (int i=0; i<N; i++) {
  sum[i] = A[i]+B[i];
  diff[i] = A[i]-B[i];
}
```

Number of memory reads reduces by a factor of two

- **Loop invariant code motion:** moves code (assignments to variables) that do not change with loop iteration

Handling Arrays

Arrays are implemented in three ways:

- **Memories:** created by a memory generator
- **Register banks**: defined in generated RTL and will go through synthesis
- **Variables:** flatten and treat them as other variables

- Read and write accesses on memories mapped for arrays must be scheduled
  - Memories: created by a memory generator

- Number of ports available in the memory limits resources for scheduling
  - When one operation accesses port other cannot access that port

```
sum1 = 0;
sum2 = 0;
for (int i=0; i<10; i++)
{
    sum1 += a[i];
    sum2 += b[i];
}
```

Handling Arrays: Hazards

```
while (...) {
  ...
  a[indx1] = x; // operation W1
  y = a[indx2]; // operation R1
  out.write(y); // some other operation
  a[indx3] = z; // operation W2
}
```

- How many minimum cycles will be needed?
- Assume: when one operation accesses port other cannot access that port
- Read/Write needs to be scheduled in different cycles (at least three cycles will be needed in the example shown)

- Assume that there are two write ports and one read port
- Yet they cannot be parallelized
  - Why?
  - Because indices indx1, indx2, indx3 could be same
  - Need to consider dependencies due to array

Hazards to consider

- RAW (read after write): R1 should be scheduled after W1
- WAR (write after read): W2 should be scheduled after R1
- WAW (write after write): W2 should be scheduled after W1

Handling Functions: Inlining

HLS tool have the option to inline a function

- Replacing the call to the function with a copy of the function body

```
inline void displayNum(int num) {
  cout << num << endl;
}

int main() {

  displayNum(5);

  displayNum(8);

  displayNum(666);
}
```

Compilation

```
inline void displayNum(int num) {
  cout << num << endl;
}

int main() {

  cout << 5 << endl;

  cout << 8 << endl;

  cout << 666 << endl;
}
```

- **Advantage:** function boundary is removed so the copy of the function can be optimized based on the caller
  - ➢ Example: constant propagation can be possible in some calls

- **Disadvantage:** DFG can grow significantly (if large function and many calls)

Handling Functions: as Resource

Non-inlined function can be treated as a resource

- Calls to the function as an operation
- Multiple calls can share a single implementation to reduce area
- Multiple instances can be created for each call to improve performance

Difference of functions with other types resources

- Latency of the function call can depend on the arguments
- Protocol between the caller and the function

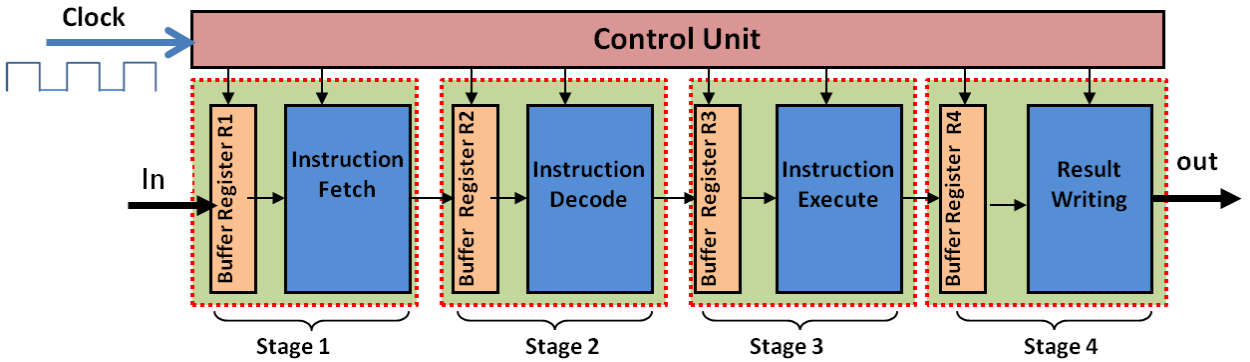
```
... // prepare arguments
notify(GO);
do {
  wait();
} while (!notified(RETURN));
... // collect results
```

```
while(true) {
  do {
    wait();
  } while (!notified(GO));
  ... // collect input arguments
  ... // perform the functionality
  ... // prepare results
  notify(RETURN);
}
```

Pipelining: Basics

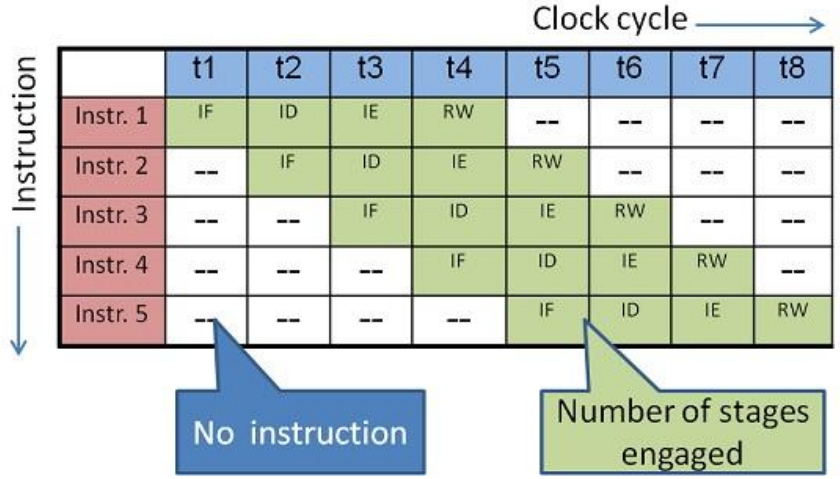
What is pipelining?

- Dividing the execution into multiple stages and each stage is handled by separate hardware
- Each stage operates concurrently like an assembly line.



What is the motivation for pipelining?

- **Non-pipelined:** each instruction must complete all steps before the next one can start (less utilization of resources)
- **Pipelined:** multiple instructions processed simultaneously (better resource utilization)
- Increase the instruction throughput
  - Throughput: number of instructions processed in a given time



- **Demerits:** increased latency, hardware complexity, area, power

HLS: Pipelining

- Pipelining enables meeting critical throughput constraints and a relaxed latency constraint
- Pipelining a loop is very common in HLS
  - ➢ Without pipelining: an iteration starts executing only after the previous iteration is completely finished
  - ➢ With pipelining: Computation of a single iteration is divided into stages

Pipelining a loop:

- An iteration starts as soon as the previous iteration completes its **first stage**
- Several iterations of the loop may be executing concurrently (each in a different stage)
- Typically, all stages take same number of cycles
  - **Initiation interval**: number of cycles needed to execute one stage
  - **Latency of a loop:** number of cycles needed to complete all stages in a loop

```
while (...) {
  while(!a1_valid.read()) wait(); // I1
  x = arr[a1.read()]; // I2
  if (f(x) > 0) y = y+1;

  while(!a2_valid.read()) wait(); // I3
  arr[a2.read()] = y; // I4
}
```

Challenges: Data dependency:

Clock-Cycle: Iteration(Stage) …

- ➢ C-N: 4 (I1) | 3 (I2) | 2 (I3) | 1 (I4)
- ➢ C-N+1: 5 (I1) | 4 (I2) | 3 (I3) | 2 (I4)
- ➢ C-N+2: 6 (I1) | 5 (I2) | 4 (I3) | 3 (I4)

- Pipeline: I2 of 4th iteration is executed before I4 of 3rd iteration
- No pipeline: I2 of 4th iteration will be executed after I4 of 3rd iteration
- If I2 of 4th iteration depends on I4 of 3rd iteration, then pipelining would be wrong

HLS Tools

Tools (HLS of C/C++ or SystemC descriptions to RTL):

- Catapult HLS from Siemens EDA
- Stratus HLS from Cadence

References

- Lavagno, L., Markov, I. L., Martin, G., & Scheffer, L. K. (Eds.). (2017). *Electronic design automation for IC system design, verification, and testing*. CRC Press.
- Micheli, G. D. (1994). Synthesis and optimization of digital circuits. McGraw-Hill Higher Education.
- S. Saurabh, "Introduction to VLSI Design Flow". Cambridge: *Cambridge University Press*, 2023.
- ChatGPT. https://chatgpt.com